Big Data Analytics and Management · TLU 2026 · Project P1

So sánh Delta Lake, Apache Iceberg và Apache Hudi dưới workload cập nhật nhiều trên cụm nhỏ cho dữ liệu IoT và khí tượng – thủy văn

Update-Heavy Workloads on Small Clusters: An Experimental Comparison of Delta Lake, Apache Iceberg and Apache Hudi for IoT and Hydro-Meteorological Data

Hà Văn Vinh · Đỗ Thị Lan

Tóm tắt kết quả

- Có dữ liệu đến trễ thì nên dùng **Iceberg Merge-on-Read**, compaction mỗi 20 batch.
- Iceberg MoR rẻ hơn 2,7–4,4 lần so với Delta và Hudi khi cùng không compaction, ở mọi $u > 0$.
- Chi phí tăng 1,3–3,8 lần ngay khi có dữ liệu trễ, rồi chỉ chênh tối đa 8% từ $u = 1\%$ đến 20% (không compaction).
- Ngay sau compaction mỗi 20 batch, Iceberg và Hudi MoR đọc gần bằng CoW (tốn 8–17 s); giữa hai lần compaction độ trễ tăng lại.
- 395/395 lần chạy khớp ground truth (một node 4 vCPU, bảng khoảng 222 MB).

Bối cảnh và vấn đề

Dữ liệu giám sát khí tượng – thủy văn là workload **cập nhật nhiều**, khác TPC-DS thiên về đọc:

- **Append liên tục:** số đo đến mỗi 10 giây đến vài phút, nạp theo micro-batch.
- **Dữ liệu đến trễ:** số đo hiệu chỉnh vài giờ sau phải ghi đè giá trị cũ.
- **Xóa theo trạm:** trạm hỏng thì xóa dữ liệu của trạm đó trong cả ngày.

Khoảng trống: LHBench và LST-Bench đo TPC-DS trên cụm đám mây lớn, trong khi cơ quan và trường ở Việt Nam thường chỉ có 1–4 máy chủ.

Câu hỏi nghiên cứu và giả thuyết

RQ	Câu hỏi	Giả thuyết
RQ1	Với $u \in \{0, 1, 5, 20\}\%$, format và chế độ ghi nào có tổng chi phí thấp nhất?	H1: ở u thấp, khác biệt nằm ở đường đọc
RQ2	Chu kỳ compaction nào cân bằng số file nhỏ và độ trễ đọc?	H2: MoR đọc chậm đến khi compaction. H3: có điểm gãy về file nhỏ
RQ3	Metadata tăng thế nào theo số commit, ảnh hưởng gì tới time travel?	Câu hỏi mô tả

Đóng góp: bộ sinh workload sensor-LST mã nguồn mở; so sánh có kiểm soát, tái lập được; bảng khuyến nghị; demo localhost.

Công trình liên quan

Công trình	Kết quả chính	Đề tài bổ sung
LHBench (CIDR 2023)	TPC-DS 3 TB: khác biệt nằm ở đọc dữ liệu	Workload ghi và sửa liên tục
LST-Bench (SIGMOD 2024)	Thêm mẫu bảo trì, time travel, độ ổn định	Phần cứng một node
AutoComp (SIGMOD 2025)	File nhỏ là vấn đề vận hành ở LinkedIn	Chu kỳ compaction là một biến
TSM-Bench (PVLDB 2023)	7 TSDB, dữ liệu thủy văn Thụy Sĩ	Dùng lại dữ liệu, thêm sửa và xóa

Ba table format, hai chế độ ghi

	Delta Lake 3.2.1	Iceberg 1.6.1	Hudi 0.15.0
Nhật ký	JSON + checkpoint mỗi 10 commit	metadata JSON → manifest	timeline + metadata table
Merge-on-Read	deletion vector	position-delete file	log file
Khóa bản ghi	không, MERGE theo vị từ	không, MERGE theo vị từ	bắt buộc, có chỉ mục
Compaction	REORG + OPTIMIZE	rewrite_data_files	compaction / clustering
Time travel	đến khi VACUUM	đến khi expire snapshot	khoảng 23 instant gần nhất

CoW: ghi lại cả file chứa dòng bị đổi. **MoR:** ghi thay đổi rẻ, hợp nhất khi đọc. **Compaction:** gộp file nhỏ, áp dụng delete.

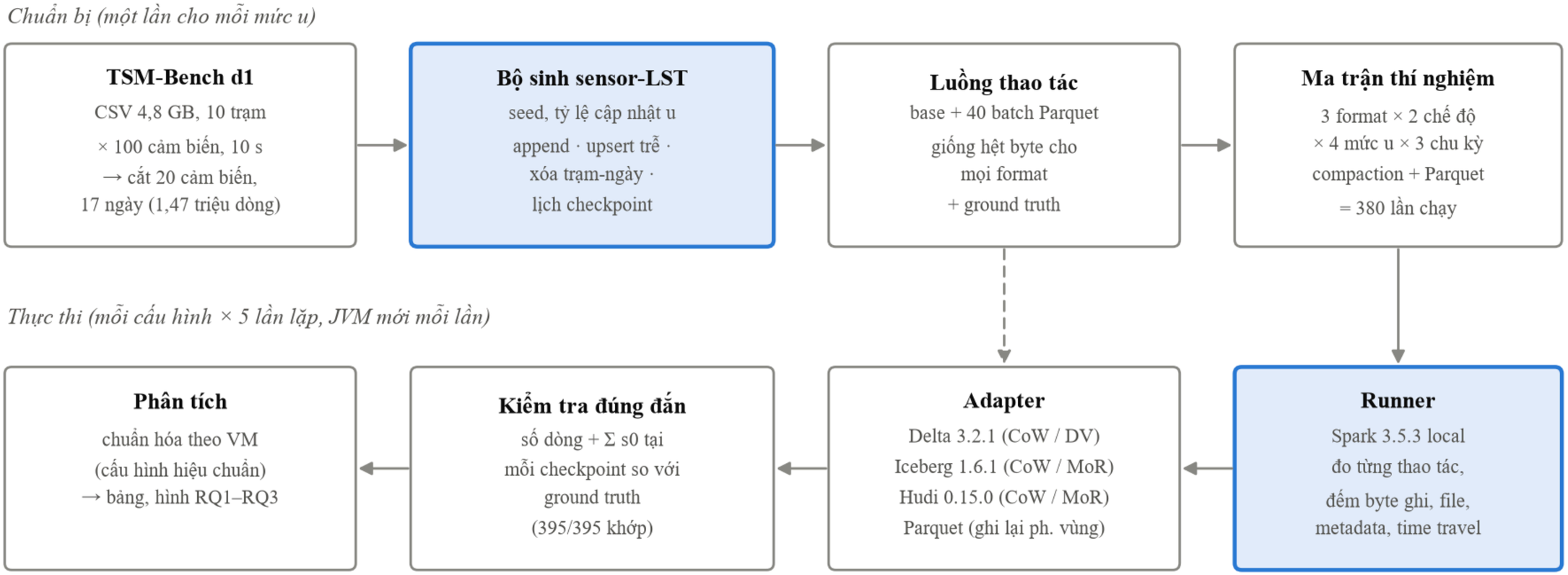
Workload sensor-LST

Mọi format nhận cùng một luồng thao tác, giống hệt byte, kèm đáp án đúng (ground truth):

1. **Nạp lịch sử:** 7 ngày, 604.800 dòng.
2. **40 micro-batch** × 6 giờ: 21.600 dòng mới + u × 21.600 bản ghi hiệu chỉnh của 72 giờ trước.
3. **Xóa:** mỗi 10 batch xóa một trạm trong một ngày.
4. **Checkpoint:** mỗi 10 batch chạy Q1–Q5, time travel, so với ground truth.

Dữ liệu: TSM-Bench d1, 10 trạm × 20 cảm biến × 17 ngày, 1,47 triệu dòng.

Kiến trúc thí nghiệm



Thiết lập và chỉ số đo

$$N = (3 \times 2 \times 4 \times 3 + 4) \times 3 = 228$$

3 format × 2 chế độ × 4 mức u × 3 chu kỳ compaction, cộng 4 baseline Parquet; mỗi cấu hình lặp 5 lần, thứ tự ngẫu nhiên.

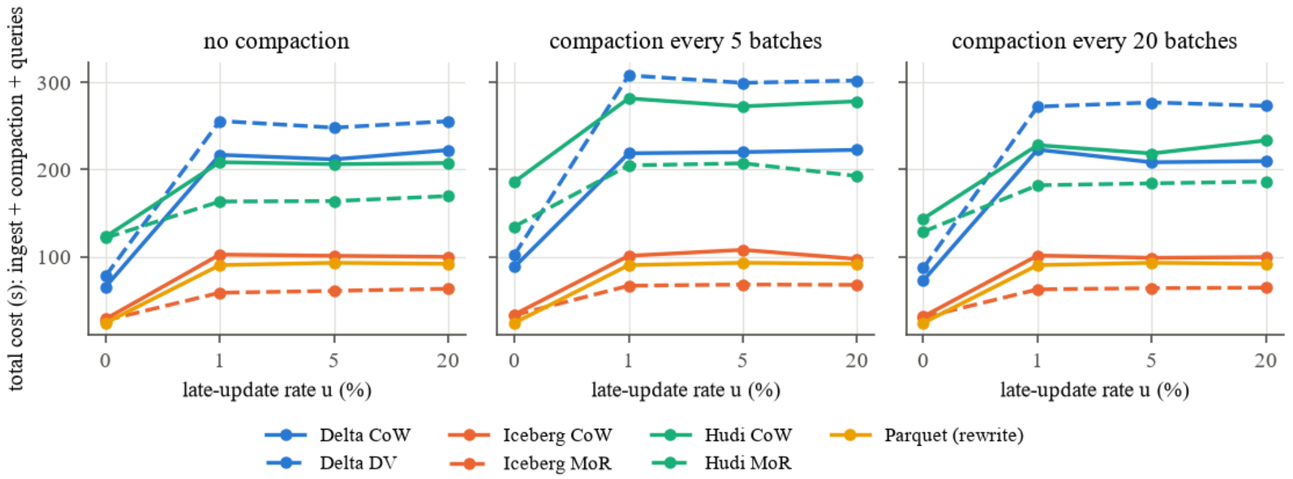
$$\mathrm{WA} = \frac{B_{\text{written}}}{B_{\text{logical}}} \qquad D = \frac{L_{\text{last}} - L_{\text{first}}}{L_{\text{first}}}$$

- Kaggle CPU: 4 vCPU, 31 GB, Spark 3.5.3 local, mỗi lần chạy một JVM mới.
- Cùng cấu hình Delta CoW, $u = 0$ mất 65–103 s trên 8 VM, nên so các mức u phải chia cho hệ số VM.

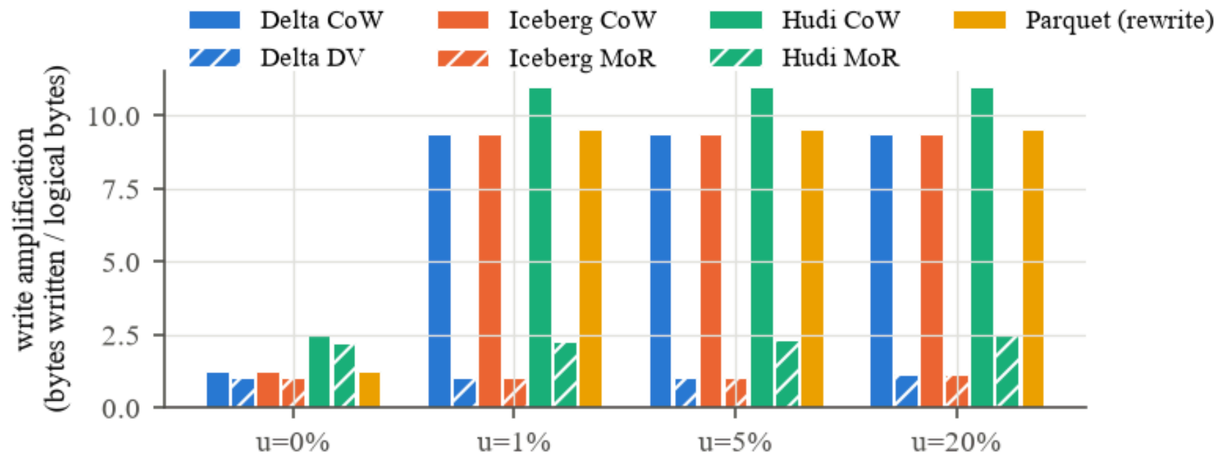
Kết quả tổng hợp

u = 20%, không compaction:

Layout	Tổng (s)	WA	Đọc (s)
Iceberg MoR	**63**	1,16	2,27
Parquet	92	9,53	0,70
Iceberg CoW	100	9,37	0,63
Hudi MoR	170	2,51	2,86
Hudi CoW	208	11,00	0,80
Delta CoW	223	9,37	1,64
Delta DV	256	1,16	4,76

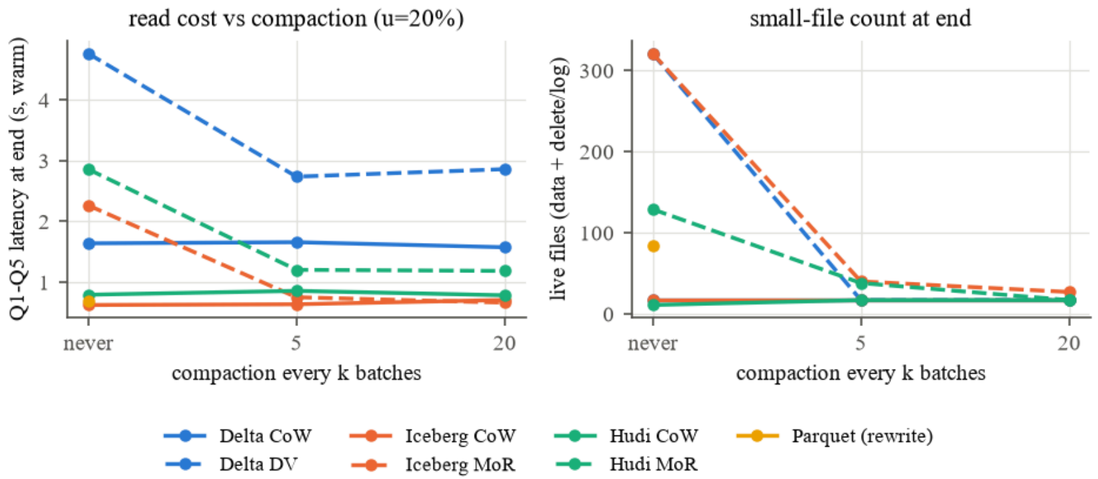


RQ1: Chi phí theo tỷ lệ cập nhật trễ



- Chi phí nhảy ở $u = 1\%$ rồi gần như đứng yên: 216 bản ghi đã chạm hầu hết file gần đây.
- CoW ghi lại 9,4 lần, dù $u = 1\%$ hay 20%.
- Delta DV và Iceberg MoR chỉ 1,0–1,2 lần.
- Byte không phải là thời gian: Delta DV ghi ít hơn CoW 8 lần nhưng nạp chậm hơn 10–13%.
- Iceberg MoR nạp nhanh gần gấp đôi Iceberg CoW.

RQ2: Chu kỳ compaction



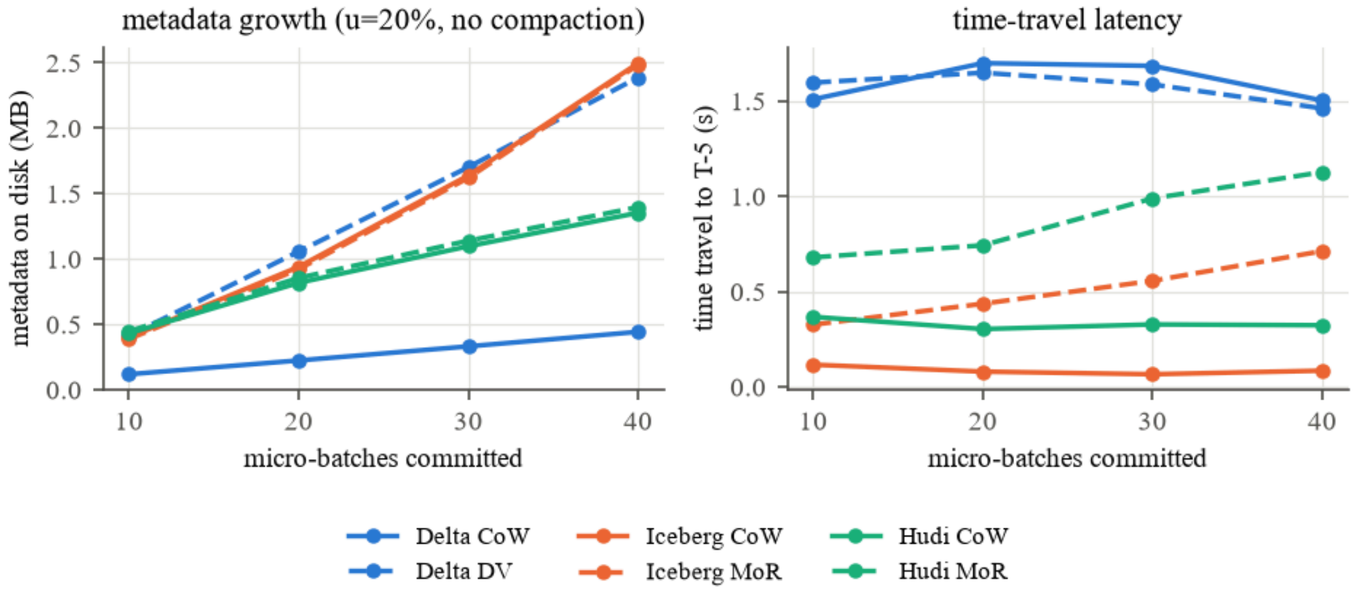
Đọc Q1–Q5, *u* = 20%	**Không compaction**	**Mỗi 20 batch**	**Mỗi 5 batch**
Iceberg MoR	2,27 s	**0,67 s**, 10 batch sau 1,29 s (tốn 8 s)	0,75 s (tốn 15 s)
Hudi MoR	2,86 s	**1,19 s**, 10 batch sau 2,13 s (tốn 17 s)	1,20 s (tốn 37 s)

RQ2: Ảnh hưởng theo từng truy vấn

Giây, $u = 20\%$	Q1	Q3	Q5
Iceberg CoW	0,10	0,28	0,10
Iceberg MoR	0,14	1,15	0,74
Iceberg MoR, mỗi 20 batch	0,12	0,27	0,09
Hudi MoR	0,14	1,28	0,95
Delta CoW	0,28	0,54	0,33
Delta DV	0,84	1,46	0,85

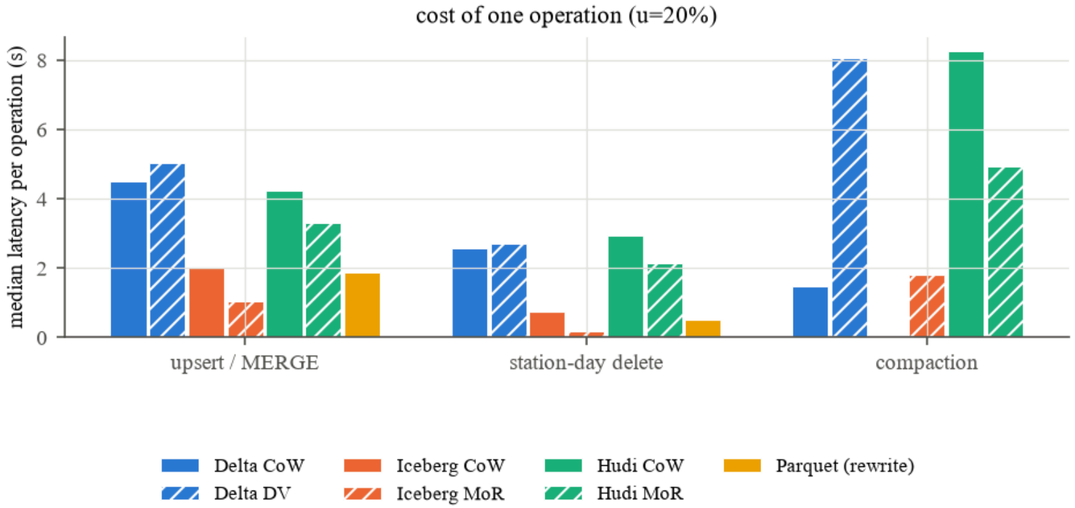
- MoR chủ yếu làm chậm truy vấn quét toàn lịch sử; với Iceberg và Hudi, truy vấn dashboard Q1 gần như không đổi.
- Q5 trên Iceberg và Hudi MoR chậm 7,7–9,5 lần so với CoW.
- Mọi truy vấn trên Delta DV chậm 2,6–3,0 lần so với Delta CoW.

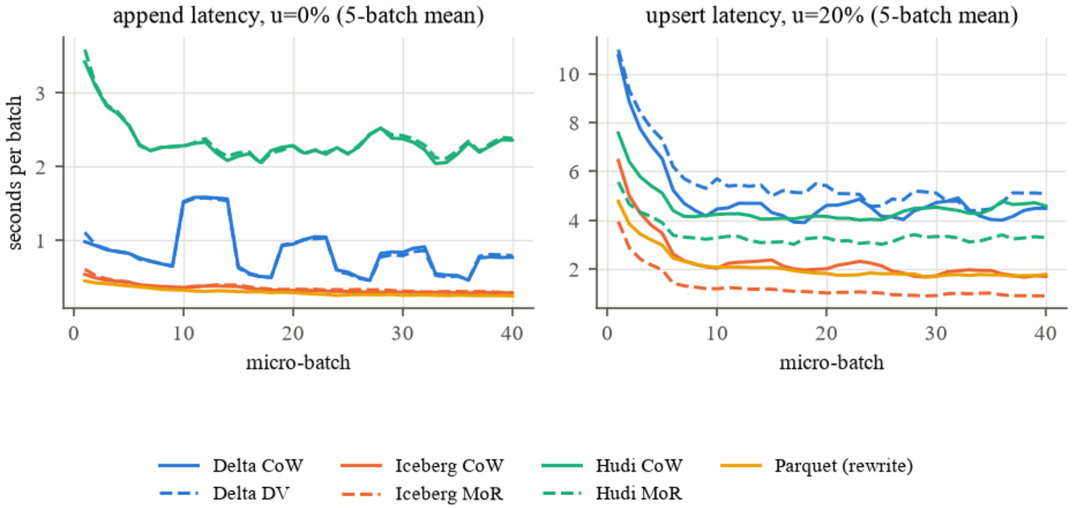
RQ3: Metadata và time travel



- Metadata tăng 11 KB/batch (Delta CoW) đến 72 KB/batch (Iceberg).
- Time travel chậm theo số file delete/log, không theo kích thước metadata.
- Hudi chỉ time travel được khoảng 23 phiên bản gần nhất.

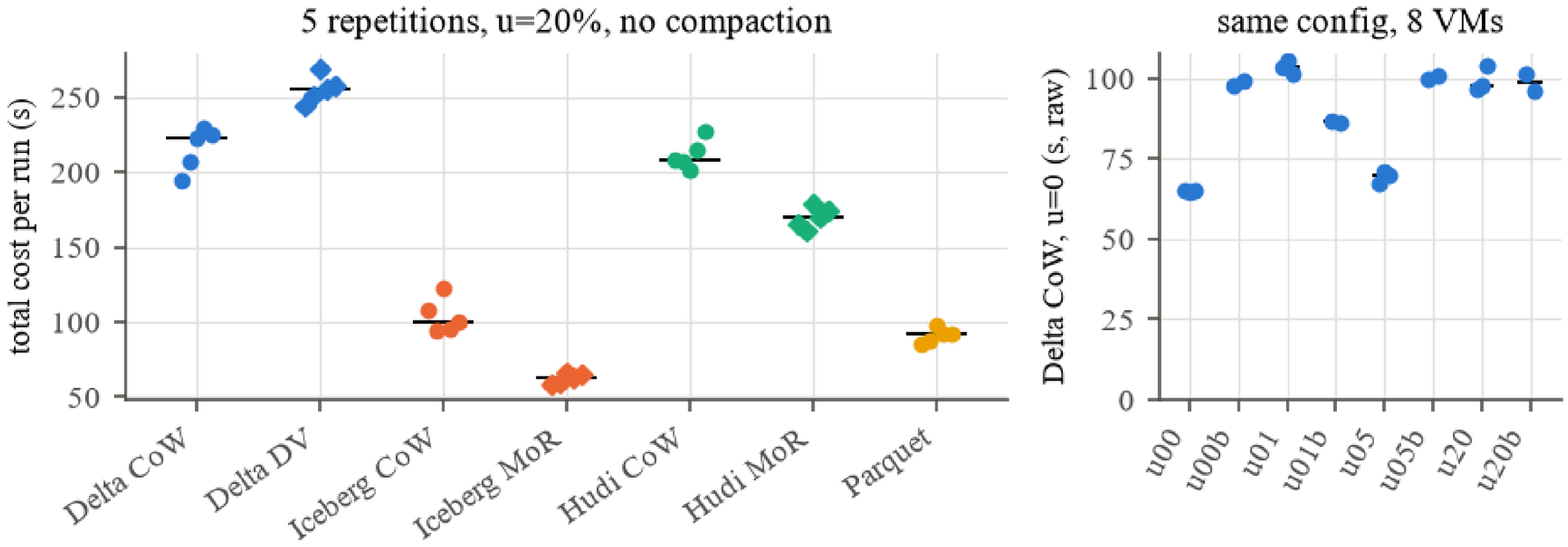
Độ trễ theo từng thao tác





- Độ trễ ghi không tăng theo kích thước bảng.
- Xóa trạm-ngày: Iceberg MoR 0,19 s; Delta và Hudi 2,2–2,9 s.
- Commit ghi checkpoint của Delta mất 2,1–5,1 s, thường chỉ 0,5 s. Append của Hudi chậm 7 lần so với Iceberg.

Độ tin cậy của số đo



- Hệ số biến thiên (CV) median 3,6%, tối đa 10,9%: các lần lặp rất ổn định.
- Chênh lệch trên 15% được xem là khác biệt thật.
- 395/395 lần chạy khớp ground truth.

Các bẫy khi dùng thực tế

Format	Vấn đề	Biểu hiện
Hudi	Khóa timestamp	bulk_insert và upsert biểu diễn khóa khác nhau, tạo bản ghi trùng im lặng
Hudi	Compaction	Mặc định chỉ lập lịch sau 5 delta commit, lệnh có thể không làm gì
Hudi	Archive	Chỉ khoảng 23 instant còn time travel được, kể cả khi đã tắt cleaner
Iceberg	Delete file “treo”	rewrite_data_files của bản 1.6 để lại 23 delete file hết tác dụng
Delta	Checkpoint đồng bộ	Mỗi commit thứ 10 chậm gấp nhiều lần commit thường
Chung	Bộ nén mặc định	Khác nhau: Iceberg zstd, Hudi gzip, Delta snappy

Hạn chế của đề tài

Hạn chế	Cách xử lý
Một node, đĩa cục bộ, chưa có object storage	Chạy lại trên cụm 3–4 node với MinIO
VM dùng chung, tốc độ chênh tới 1,58 lần	Lặp 5 lần, xen kẽ, hiệu chuẩn theo VM
Bảng khoảng 222 MB, chưa chạy TPC-DS SF10–100	Nêu rõ phạm vi; bổ sung TPC-DS SF10
Giá trị d1 gần ngẫu nhiên, nén kém	Chỉ so sánh tỷ lệ giữa các format
Phụ thuộc phiên bản và cấu hình	Công bố mọi thiết lập trong repo
Chỉ một engine là Spark	Đọc đa engine với Trino, DuckDB

Kết luận và khuyến nghị

Tình huống	Nên dùng	Compaction
Có dữ liệu trễ (*u* = 1–20%)	**Iceberg MoR**	**mỗi 20 batch; đọc nhiều: mỗi 5**
Chỉ append, cần ACID, lịch sử	Iceberg	mỗi 20 batch
Chỉ append, không cần ACID	Parquet thuần	không cần
Đã dùng Delta Lake	Delta CoW, tắt DV	tùy chọn
Đã dùng Hudi	Hudi MoR	mỗi 20 batch

Hướng tiếp theo: cụm 3–4 node với MinIO; đọc đa engine; compaction theo ngân sách; bổ sung TPC-DS SF10.

Cảm ơn thầy và các bạn

Tiếp theo: demo time travel trực tiếp, rồi hỏi đáp

github.com/vinhha195-oss/lakehouse-p1